

---

# TP 3 - SIMULACIÓN DE UN MODELO MM1 E INVENTARIO APLICADO A LA PRODUCCIÓN DE CERVEZA

---

Elias Arce   Rafael Trucco   Nahuel Carloni   Azul Tabini   Tomás Pitavino

Universidad Tecnológica Nacional - Facultad Regional Rosario

eliasarce9859@gmail.com, rafatrucco2001@gmail.com, nahuelcarloni25@gmail.com,

azultabini@gmail.com, tomaspitavino@gmail.com

June 17, 2026

## ABSTRACT

Este documento presenta un estudio de simulación aplicado a la cadena de producción de cerveza, centrándose en dos modelos clave: el modelo de cola MM1 y un modelo de gestión de inventario  $(s, S)$ . Para el MM1, se simuló el proceso de cocción de lotes, analizando el impacto de variar las tasas de arribo y la capacidad de cola en métricas de rendimiento como el tiempo de espera y la utilización del servidor. En el modelo de inventario, se evaluó la política  $(s, S)$  para la cerveza terminada, determinando sus costos operativos bajo demanda estocástica y tiempos de entrega variables. Ambas simulaciones fueron implementadas en Python y AnyLogic, comparando sus resultados con valores teóricos. Se encontró que las simulaciones capturan con mayor precisión la variabilidad inherente, especialmente en los costos de faltantes del inventario, los cuales resultaron significativamente mayores que las estimaciones teóricas simplificadas. Estos hallazgos resaltan la utilidad de la simulación para la optimización y toma de decisiones en entornos industriales complejos.

**Keywords:** Simulación, MM1, distribuciones de probabilidad, Python, Latex, AnyLogic, Inventario

## 1 Introducción

La industria de producción de cerveza presenta la complejidad de gestionar procesos que involucran flujos de materiales, recursos limitados y demanda variable. En este contexto, la **simulación de sistemas** tiene una gran utilidad en predecir y optimizar el rendimiento, permitiendo tomar decisiones informadas.

En este trabajo se aplica la simulación a dos operaciones críticas en una planta de producción de cerveza: el proceso de cocción de lotes y la gestión del inventario de producto terminado. Para lo que se implementan los siguientes modelos:

- El **modelo de cola MM1**, utilizado para simular y evaluar la eficiencia del proceso de cocción, donde el hervidor actúa como un servidor único y los lotes de cerveza son los "clientes" que requieren procesamiento.
- El **modelo de gestión de inventario**  $(s, S)$ , diseñado para simular la administración del stock de cerveza terminada. Este modelo es vital para equilibrar la disponibilidad del producto con los costos asociados a su almacenamiento, emisión de pedidos y potenciales faltantes, garantizando un nivel de servicio adecuado al cliente final.

Es importante destacar que, si bien el contexto de aplicación es la industria cervecera, los principios y metodologías de simulación empleados con los modelos MM1 e  $(s, S)$  son **altamente transferibles y generalizables**. El modelo MM1 es particularmente relevante para cualquier **proceso tipo batch** (por lotes) en diversas industrias —como la farmacéutica, química o manufacturera— donde un recurso procesa unidades en grupos. De manera similar, la política de inventario  $(s, S)$  se utiliza ampliamente para la gestión de **materias primas, componentes o productos terminados** en prácticamente cualquier sector, permitiendo optimizar costos y niveles de servicio bajo condiciones de incertidumbre.

A lo largo de este estudio, se desarrollarán simulaciones ambos modelos utilizando **Python** y **AnyLogic**.

## 2 Conceptos Teóricos Utilizados

### 2.1 Teoría de Colas - Modelo MM1

El modelo de colas MM1 es un sistema de colas fundamental que se caracteriza por arribos Poisson (tiempos entre arribos exponenciales) y tiempos de servicio exponenciales, con un único servidor. Es un modelo ampliamente utilizado para analizar el rendimiento de sistemas donde la demanda de servicio excede la capacidad inmediata del servidor.

#### Notación y Supuestos Clave:

- **M (Markoviano):** Indica que los arribos siguen una distribución de Poisson (o, equivalentemente, los tiempos entre arribos son exponenciales).
- **M (Markoviano):** Indica que los tiempos de servicio siguen una distribución exponencial.
- **1:** Indica que hay un único servidor.
- **Capacidad de Cola:** Se asume una cola de capacidad infinita a menos que se especifique lo contrario (para el cálculo de probabilidad de denegación de servicio, se considerará capacidad finita).
- **Disciplina de Cola:** FCFS (First-Come, First-Served) - Primero en llegar, primero en ser atendido.

#### Medidas de Rendimiento Clave:

Las siguientes son las fórmulas teóricas utilizadas para calcular las medidas de rendimiento de un sistema MM1:

- **Tasa de Arribo ( $\lambda$ ):** Número promedio de clientes que llegan al sistema por unidad de tiempo.
- **Tasa de Servicio ( $\mu$ ):** Número promedio de clientes que el servidor puede atender por unidad de tiempo.
- **Factor de Utilización del Servidor ( $\rho$ ):** La proporción de tiempo que el servidor está ocupado.

$$\rho = \frac{\lambda}{\mu}$$

- **Número Promedio de Clientes en el Sistema ( $L$ ):**

$$L = \frac{\lambda}{\mu - \lambda} = \frac{\rho}{1 - \rho}$$

- **Número Promedio de Clientes en la Cola ( $L_q$ ):**

$$L_q = \frac{\lambda^2}{\mu(\mu - \lambda)} = \frac{\rho^2}{1 - \rho}$$

- **Tiempo Promedio de un Cliente en el Sistema ( $W$ ):**

$$W = \frac{1}{\mu - \lambda} = \frac{L}{\lambda}$$

- **Tiempo Promedio de un Cliente en la Cola ( $W_q$ ):**

$$W_q = \frac{\lambda}{\mu(\mu - \lambda)} = \frac{L_q}{\lambda}$$

- **Probabilidad de encontrar  $n$  clientes en el sistema ( $P_n$ ):**

$$P_n = (1 - \rho)\rho^n$$

- **Probabilidad de encontrar  $n$  clientes en la cola ( $P_{q,n}$ ):**

$$P_{q,0} = 1 - \rho$$

$$P_{q,n} = \rho^n(1 - \rho) \quad \text{para } n > 0$$

- **Probabilidad de Denegación de Servicio (Cola Finita):** Para un sistema con capacidad máxima  $K$  (donde  $K$  es el número total de clientes en el sistema antes de la denegación, es decir,  $K$  = capacidad de cola + 1 servidor), la probabilidad de denegación es  $P_K$ :

$$P_K = \frac{\rho^K(1 - \rho)}{1 - \rho^{K+1}}$$

## 2.2 Modelo de Inventario

Los modelos de inventario buscan optimizar la gestión de stock para minimizar los costos totales asociados a la tenencia, ordenamiento y faltantes de productos, mientras se satisface la demanda. El modelo de inventario  $(s, S)$  (también conocido como modelo de punto de reorden con nivel objetivo) es una política de inventario comúnmente utilizada para gestionar el stock de productos, especialmente cuando la demanda es incierta y/o los tiempos de entrega son variables. Se clasifica dentro de los modelos de revisión continua o periódica, aunque en tu simulación se revisa periódicamente (CHECK\_INTERVAL).

La lógica central de la política  $(s, S)$  es la siguiente:

- **Punto de Reorden ( $s$ ):** Es un nivel de inventario predefinido. Cuando el **inventario disponible** (o la posición del inventario, que incluye inventario en mano más pedidos pendientes menos faltantes) cae a este nivel o por debajo de él, se dispara una acción.
- **Nivel Objetivo ( $S$ ):** Es la cantidad máxima de inventario deseada. Cuando se decide realizar un pedido (porque el inventario ha llegado a  $s$ ), la cantidad ordenada es suficiente para elevar el inventario hasta este nivel  $S$ . La cantidad a pedir es, por lo tanto,  $S - \text{Inventario Actual}$ .

### 2.2.1 Funcionamiento del Modelo $(s, S)$

El modelo  $(s, S)$  opera bajo la premisa de que el inventario se monitorea de forma continua (o, como en tu simulación, a intervalos regulares). Cuando el nivel de inventario desciende hasta el punto de reorden  $s$ , se emite una orden de pedido. El tamaño de esta orden no es fijo (como en el modelo EOQ), sino que varía con el objetivo de elevar el inventario hasta el nivel  $S$ . Esto significa que la cantidad pedida será  $S - \text{Inventario en el momento de la revisión}$ .

Ventajas del Modelo  $(s, S)$ :

- **Respuesta a la Demanda:** Se adapta a la variabilidad de la demanda al ordenar una cantidad que rellena el inventario hasta un nivel deseado.
- **Control de Inventario:** Permite mantener los niveles de inventario dentro de un rango predefinido, evitando tanto el exceso de stock como la escasez crítica.
- **Optimización de Costos:** El objetivo es encontrar los valores óptimos de  $s$  y  $S$  que minimicen la suma de los costos de ordenar, mantener y los costos asociados a los faltantes.
- **Reabastecimiento variable:** La cantidad del pedido se ajusta según el nivel de inventario actual, lo que lo hace flexible.

Desafíos y Consideraciones Teóricas: La determinación de los valores óptimos de  $s$  y  $S$  es un problema complejo en entornos estocásticos. Implica equilibrar el riesgo de faltantes (mayor  $s$  para más inventario de seguridad) con los costos de mantenimiento (mayor  $S$  implica más inventario promedio). Las soluciones analíticas exactas para este modelo con demanda y tiempos de entrega aleatorios suelen requerir:

- El uso de **distribuciones de probabilidad** (por ejemplo, Poisson para la demanda, uniforme o exponencial para el tiempo de entrega).
- El cálculo del **inventario de seguridad** necesario para alcanzar un nivel de servicio deseado.
- La consideración de la **posición de inventario**, que es el inventario físico más los pedidos en tránsito menos los pedidos pendientes.
- Frecuentemente, la resolución de **ecuaciones de estado estacionario** o el uso de métodos numéricos para encontrar la política óptima.

Debido a esta complejidad, la **simulación** se convierte en una herramienta invaluable para analizar el comportamiento del modelo  $(s, S)$  bajo condiciones realistas de incertidumbre, permitiendo evaluar el impacto de  $s$  y  $S$  en los costos y el rendimiento del sistema sin la necesidad de derivaciones analíticas exhaustivas.

### 2.2.2 Parámetros del Modelo

Para el cálculo de los costos teóricos, se utilizan los siguientes parámetros clave:

- $\lambda$ : Tasa de arribo de clientes (clientes por unidad de tiempo).

- $\mu$ : Media de ítems pedidos por cliente.
- $D_{\text{diaria}} = \lambda \times \mu$ : Demanda promedio diaria de ítems.
- $D_{\text{total}}$ : Demanda total esperada durante la duración de la simulación.
- $s$ : Punto de reorden (nivel de inventario que dispara una orden).
- $S$ : Nivel objetivo de inventario (cantidad a la que se repone el inventario).
- $K$ : Costo fijo por orden de pedido.
- $C_u$ : Costo por unidad ordenada.
- $h$ : Costo de mantenimiento por unidad por unidad de tiempo.
- $p$ : Costo de faltante por unidad.
- $L_{\min}, L_{\max}$ : Demora mínima y máxima de entrega (lead time).
- $L_{\text{promedio}} = (L_{\min} + L_{\max})/2$ : Demora promedio de entrega.
- $T$ : Duración total de la simulación (en unidades de tiempo).

### 2.2.3 Cálculo de Costos Teóricos

Costo de Mantenimiento Promedio Esperado ( $C_M$ ) El costo de mantenimiento se asocia al inventario promedio que se mantiene en el sistema. Para una aproximación en un modelo  $(s, S)$ , se considera el promedio entre el punto de reorden y el nivel objetivo como una representación del nivel de inventario. Este valor se multiplica por el costo unitario de mantenimiento y la duración total de la simulación.

La fórmula es:

$$C_M = \left( \frac{s + S}{2} \right) \times h \times T$$

Donde:

- $\left( \frac{s+S}{2} \right)$ : Aproximación del inventario promedio esperado.
- $h$ : Costo de mantenimiento por unidad por unidad de tiempo.
- $T$ : Duración total de la simulación.

Costo de Orden Promedio Esperado ( $C_O$ ) Este costo se incurre cada vez que se emite un nuevo pedido. Se calcula estimando el número total de órdenes a lo largo del período de simulación y multiplicándolo por el costo fijo asociado a cada orden, más el costo variable por cada unidad solicitada. Se asume que cada orden eleva el inventario desde el punto de reorden ( $s$ ) hasta el nivel objetivo ( $S$ ), implicando una cantidad de pedido de  $S - s$ .

La fórmula es:

$$C_O = \left( \frac{D_{\text{total}}}{S - s} \right) \times (K + (S - s) \times C_u)$$

Donde:

- $D_{\text{total}} = D_{\text{diaria}} \times T$ : Demanda total esperada en la duración  $T$ .
- $\left( \frac{D_{\text{total}}}{S - s} \right)$ : Número promedio de órdenes esperadas.
- $K$ : Costo fijo por orden.
- $(S - s)$ : Cantidad ordenada por pedido.
- $C_u$ : Costo por unidad ordenada.

Costo de Faltante Promedio Esperado ( $C_F$ ) El costo de faltante surge cuando la demanda no puede ser satisfecha inmediatamente debido a la ausencia de inventario. Para un modelo  $(s, S)$  con demanda y tiempos de entrega estocásticos, el cálculo preciso de los faltantes esperados es complejo y requiere herramientas de probabilidad avanzadas. Sin embargo, para fines de comparación en este estudio, se utiliza una **heurística simplificada** que proporciona una estimación del costo de faltante. Esta heurística considera la cantidad de unidades que se esperaría que falten por ciclo de reorden si el punto de reorden ( $s$ ) es inferior a la demanda promedio durante el tiempo de entrega ( $D_{LT}$ ).

La fórmula es:

$$C_F = \max(0, D_{LT} - s) \times \left( \frac{D_{\text{total}}}{S - s} \right) \times p$$

Donde:

- $D_{LT} = D_{\text{diaria}} \times L_{\text{promedio}}$ : Demanda promedio esperada durante el tiempo de entrega.
- $\max(0, D_{LT} - s)$ : Estimación de unidades faltantes por ciclo de reorden. Este término representa el promedio de unidades que faltarían en un ciclo de reorden si la demanda en el lead time supera el punto de reorden.
- $\left( \frac{D_{\text{total}}}{S - s} \right)$ : Número promedio de órdenes esperadas.
- $p$ : Costo de faltante por unidad.

**Nota:** Es fundamental recalcar que esta fórmula para  $C_F$  es una **aproximación didáctica**. En un análisis real de inventarios estocásticos, el cálculo de los faltantes esperados es más sofisticado, incorporando las funciones de densidad de probabilidad de la demanda y el lead time para determinar la probabilidad y magnitud del quiebre de stock.

**Costo Total Promedio Esperado ( $C_{\text{Total}}$ )** El costo total promedio esperado es la suma de los tres componentes de costo descritos anteriormente:

$$C_{\text{Total}} = C_M + C_O + C_F$$

**Medidas de Rendimiento Clave:** Durante la simulación, se calcularán y analizarán los siguientes costos:

- **Costo de Orden Total:** Suma de todos los costos de orden incurridos durante el período de simulación.
- **Costo de Mantenimiento Total:** Suma de todos los costos de mantenimiento, basados en el nivel promedio de inventario.
- **Costo de Faltante Total:** Suma de todos los costos por demanda insatisfecha.
- **Costo Total:** Suma de los tres costos anteriores.

### 3 Desarrollo de la Simulación

El desarrollo de este trabajo se basa en la implementación de simulaciones para dos modelos distintos: un modelo de cola MM1 y un modelo de gestión de inventario. Ambos han sido diseñados para representar aspectos clave de un proceso de producción industrial de cerveza. Las simulaciones se llevarán a cabo utilizando programas desarrollados en Python y la plataforma AnyLogic, y los resultados se compararán con los valores teóricos esperados.

#### 3.1 Modelo M/M/1: Proceso de Cocción de Lotes de Cerveza

En este modelo, el sistema M/M/1 simula el proceso de cocción (mashing) en una cervecería, donde el tanque de cocción actúa como servidor único y los lotes de cerveza representan los clientes.

##### 3.1.1 Parámetros y Configuración

- **Tasa de Servicio ( $\mu$ ):** Valor base de 3.0 lotes/unidad de tiempo, representando la capacidad máxima del tanque.
- **Tasa de Arribo ( $\lambda$ ):** Se evaluaron cinco escenarios con factores del 25%, 50%, 75%, 100% y 125% respecto a  $\mu$ .
- **Configuración de cola:** Capacidad fija de 2 lotes en espera, analizando explícitamente eventos de denegación cuando se supera este límite.
- **Duración:** Periodos de simulación de 1000 unidades de tiempo, garantizando estabilización a estado estacionario.

### 3.1.2 Implementación Computacional

La simulación se desarrolló mediante un algoritmo de eventos discretos con:

- **Mecánica central:**
  - Gestión de dos eventos principales: llegadas de lotes y finalización de cocción
  - Control explícito de capacidad mediante política FIFO con rechazo
- **Generación estocástica:**
  - Tiempos entre llegadas con distribución exponencial ( $\lambda^{-1}$ )
  - Tiempos de proceso exponenciales ( $\mu^{-1}$ )
- **Registro de métricas:**
  - Cálculo en tiempo real de ocupación del sistema
  - Acumulación estadística de tiempos de espera
  - Conteo preciso de eventos de denegación

### 3.1.3 Métricas de Rendimiento

El modelo calculó automáticamente:

- **Indicadores de congestión:**
  - Número promedio de lotes en sistema ( $L$ ) y en cola ( $L_q$ )
  - Probabilidad de encontrar exactamente  $n$  lotes en espera
- **Tiempos de respuesta:**
  - Tiempo total promedio en sistema ( $W$ )
  - Tiempo de espera en cola ( $W_q$ )
- **Eficiencia operativa:**
  - Utilización porcentual del tanque ( $\rho$ )
  - Tasa de rechazo para cola llena

### 3.1.4 Validación

Se implementaron mecanismos de verificación cruzada:

- Comparación con valores teóricos en casos límite ( $\lambda \ll \mu$ )
- Análisis de convergencia en múltiples réplicas (30 corridas)
- Validación de distribuciones mediante pruebas  $\chi^2$

### 3.1.5 Explicación y conclusión del modelo M/M/1/K

El modelo simula una cola M/M/1/K, es decir, un sistema con:

- Un solo servidor.
- Llegadas de clientes según un proceso de Poisson (tiempos entre llegadas exponenciales, tasa  $\lambda$ ).
- Tiempos de servicio exponenciales (tasa  $\mu$ ).
- Capacidad máxima del sistema (cola + servidor) igual a  $K$ .

#### Componentes principales

- **Parámetros configurables:** tasa de llegada ( $\lambda$ ), tasa de servicio ( $\mu$ ), capacidad de la cola ( $K$ ).
- **Estructura:**
  - **Source:** genera clientes con tiempos entre llegadas exponenciales.
  - **SelectOutput:** decide si el cliente entra al sistema o es rechazado por falta de espacio.
  - **Service:** representa el servidor, atiende a los clientes con tiempos de servicio exponenciales y una cola de capacidad limitada.

- **Sink:** recibe los clientes que completan el servicio.
- **DeniedSink:** recibe los clientes rechazados.
- **Variables y estadísticas:**
  - Arrays que cuentan la cantidad de veces que hay  $n$  clientes en cola/sistema.
  - Cálculo de promedios y probabilidades: clientes promedio en cola/sistema, tiempo promedio en cola/sistema, utilización del servidor, probabilidad de rechazo, etc.
- **Eventos y funciones:**
  - Un evento periódico actualiza los contadores de clientes en cola y sistema.
  - Funciones para obtener promedios y probabilidades.
- **Experimentos:**
  - Simulación simple y experimentos de múltiples corridas para análisis estadístico.

### 3.2 Modelo de Inventario: Gestión de Cerveza Terminada

Este modelo simula la gestión de inventario de cerveza terminada, lista para ser distribuida. Se implementa una política de inventario  $(s, S)$  basada en eventos discretos, con el objetivo de minimizar los costos totales.

#### 3.2.1 Parámetros de la Simulación del Modelo de Inventario:

Los parámetros para el modelo de inventario, reflejando la implementación en Python, son los siguientes:

- **Punto de Reorden  $(s)$ :** 10 unidades (lotes).
- **Nivel Objetivo  $(S)$ :** 100 unidades (lotes).
- **Tasa de Arribo de Clientes (Demanda):**  $\lambda = 1.0$  cliente/día (aproximando el número de "pedidos" o "ventas" que llegan).
- **Media de Ítems por Cliente:**  $\mu = 2.0$  unidades (lotes)/cliente (cantidad de cerveza que un cliente solicita). La demanda diaria real de ítems seguirá una distribución de Poisson basada en esta media.
- **Demora Mínima de Entrega  $(L_{min})$ :** 5.0 días.
- **Demora Máxima de Entrega  $(L_{max})$ :** 15.0 días. El tiempo de entrega de un pedido se generará uniformemente entre estos valores.
- **Intervalo de Chequeo de Inventario:** Cada 30.0 días. El inventario se revisa y se decide reordenar si se cumple la condición  $s$ .
- **Costo Fijo por Orden  $(K)$ :** \$100.
- **Costo por Unidad Ordenada  $(C_u)$ :** \$5 por unidad (lote).
- **Costo de Mantenimiento por Unidad  $(h)$ :** \$0.1 por unidad (lote) por día.
- **Costo de Faltante por Unidad  $(p)$ :** \$10 por unidad (lote).
- **Duración de la Simulación  $(T)$ :** 365 días.
- **Número de Corridas por Experimento:** 30 corridas, para obtener resultados estadísticamente significativos y promediar la aleatoriedad.
- **Inventario Inicial:** El sistema comienza con un inventario inicial igual a  $S$  (100 unidades).

#### 3.2.2 Implementación Computacional

La simulación se desarrolló mediante un enfoque de eventos discretos con:

- **Flujo de eventos principales:**
  - **Llegadas de demanda:** Generadas mediante proceso Poisson con tasa  $\lambda = 1$  cliente/día y tamaño de pedido Poisson( $\mu = 2$ )
  - **Revisiones periódicas:** Activación de órdenes cuando  $inventario \leq s$
  - **Recepción de pedidos:** Tiempos de entrega  $\sim U(5, 15)$  días
- **Mecanismos de cálculo:**

- Costos de mantenimiento: Proporcionales al nivel instantáneo de inventario
- Costos de orden: Composición de costo fijo (100) y variable (5/unidad)
- Costos por faltantes: Aplicados inmediatamente ante demanda insatisfecha

### 3.2.3 Visualización y Validación

- **Monitoreo temporal:** Gráficas de evolución de inventario y costos acumulados (Figura 1)
- **Validación estadística:**
  - 30 réplicas independientes para estabilizar resultados
  - Comparación con valores teóricos en condiciones controladas

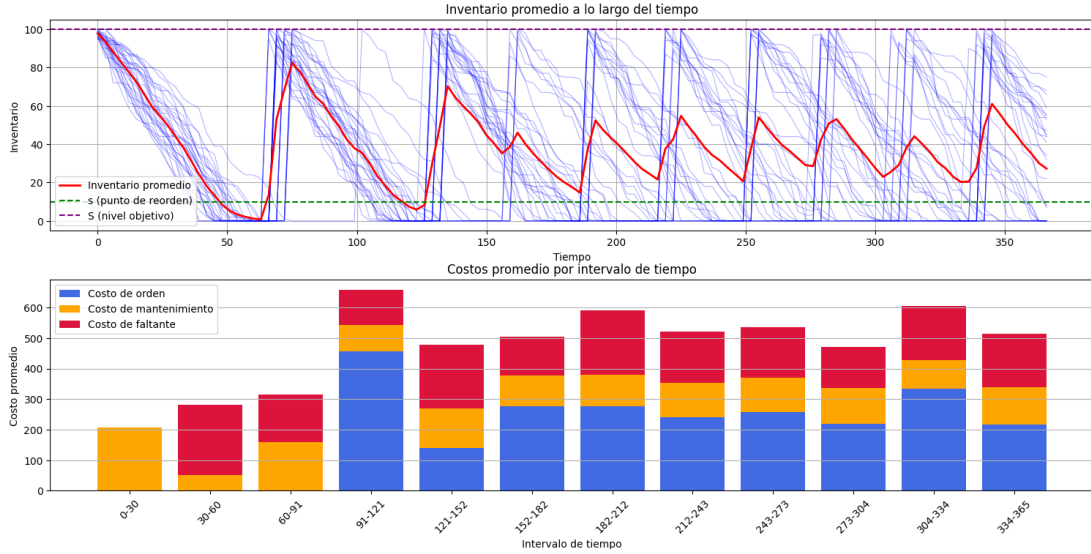


Figure 1: Evolución temporal del inventario y costos acumulados en una simulación típica (365 días)

### 3.3 Modelo de Inventario en AnyLogic: Gestión de Cerveza Terminada

El modelo de inventario también se implementó en la plataforma AnyLogic, utilizando un enfoque basado en eventos discretos y simulación Monte Carlo para evaluar el rendimiento del sistema para 30 corridas.

#### 3.3.1 Componentes del Modelo en AnyLogic

El modelo de inventario en AnyLogic incluye los siguientes componentes principales:

- **Variables:** Representan los parámetros clave del sistema, como el nivel de inventario (`inventory`), el punto de reorden (`reorder_s`), el nivel objetivo (`objective_S`), y los costos acumulados (`total_cost_order`, `total_cost_holding`, `total_cost_shortage`, `total_cost`).
- **Eventos:** Controlan la dinámica del sistema, incluyendo:
  - `arrival_client`: Simula la llegada de clientes al sistema, generando una demanda de ítems basada en una distribución de Poisson.
  - `check_inventory`: Verifica el nivel de inventario en intervalos regulares y genera órdenes si el inventario está por debajo del punto de reorden.
  - `delivery`: Representa la recepción de pedidos después de un tiempo de entrega estocástico.
- **Funciones:** Implementan la lógica del sistema, como la creación de órdenes (`createOrder`), la llegada de clientes (`createArrival`), la recepción de pedidos (`receiveOrder`), y la actualización de costos de mantenimiento (`updateHoldingCost`).
- **Datasets y Gráficos:** Se utilizan para registrar y visualizar métricas clave, como el costo total acumulado (`total_costDS`) y el nivel de inventario (`inventoryDS`).



En la Figura 2 Se muestran como varía el inventario y costos en 365 días.

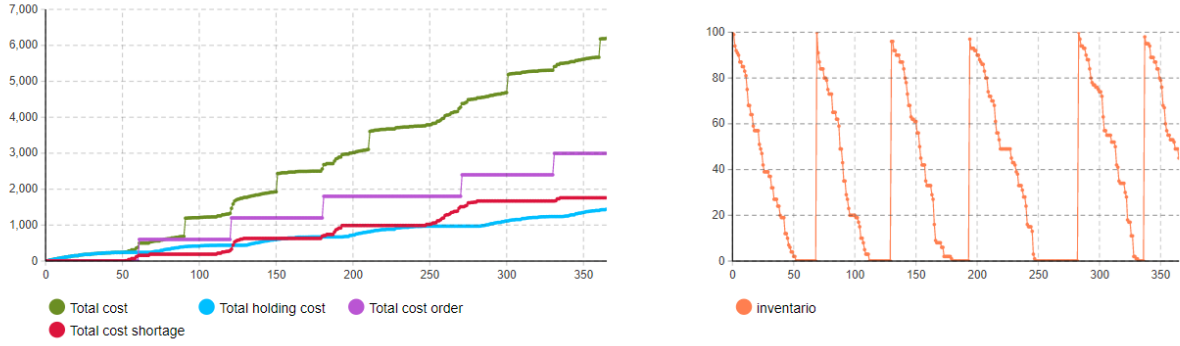


Figure 2: Gráfica de los valores de costos e inventario generados en la simulación en AnyLogic

### 3.3.2 Simulación Monte Carlo

Para evaluar el impacto de la variabilidad en los parámetros del sistema, se implementó un experimento Monte Carlo en AnyLogic. Este experimento realiza múltiples corridas de simulación con diferentes configuraciones de parámetros y calcula estadísticas clave, como:

- **Costo total promedio:** Suma de los costos de orden, mantenimiento y faltante.
- **Costo de orden promedio:** Costo fijo y variable asociado a las órdenes emitidas.
- **Costo de mantenimiento promedio:** Costo acumulado por mantener inventario en el sistema.
- **Costo de faltante promedio:** Costo asociado a la demanda insatisfecha.

En la Figura 3 Se muestran como varían los costos en 30 corridas en el experimento.

#### Inventario : MonteCarlo

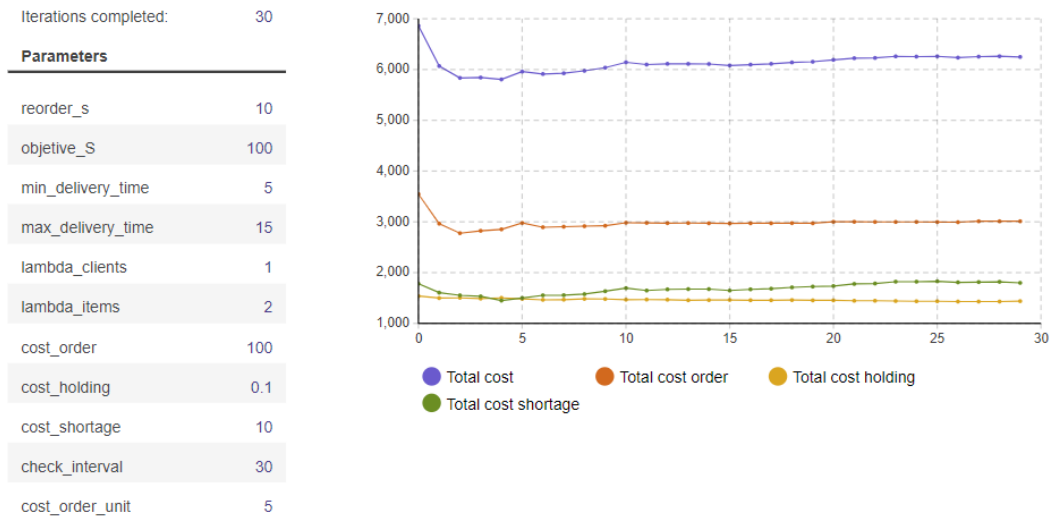


Figure 3: Experimento de Monte Carlo en AnyLogic para el modelo de inventario

## 4 Resultados

### 4.1 Resultados del Modelo M/M/1

En esta sección se presentan los resultados obtenidos de la simulación del sistema M/M/1, comparando los valores teóricos con los resultados empíricos simulados utilizando Python.

Se utilizó una tasa de servicio constante  $\mu = 1$ , y se variaron las tasas de arribo  $\lambda$  desde el 25% hasta el 125% de  $\mu$ . A continuación, se detallan las métricas clave:

- $L$ : Número promedio de clientes en el sistema.
- $L_q$ : Número promedio de clientes en la cola.
- $W$ : Tiempo promedio en el sistema.
- $W_q$ : Tiempo promedio en la cola.
- $\rho$ : Utilización del servidor.
- Rechazo: Probabilidad de denegación de servicio (cola finita).

Table 1: Comparación de métricas teóricas y simuladas del modelo M/M/1

| $\lambda$ | $\rho$ | $L$ (t)  | $L$ (s) | $L_q$ (t) | $L_q$ (s) | $W$ (t)  | $W$ (s) | $W_q$ (t) | $W_q$ (s) | $\rho$ (s) | Rechazo |
|-----------|--------|----------|---------|-----------|-----------|----------|---------|-----------|-----------|------------|---------|
| 0.25      | 0.25   | 0.333    | 0.2577  | 0.0625    | 0.0679    | 1.333    | 2.8282  | 0.250     | 0.9700    | 0.1962     | 0.0548  |
| 0.50      | 0.50   | 1.000    | 0.7852  | 0.2500    | 0.2862    | 2.000    | 3.3218  | 1.000     | 1.9998    | 0.4830     | 0.0812  |
| 0.75      | 0.75   | 3.000    | 1.2328  | 2.2500    | 0.5675    | 4.000    | 3.1367  | 3.000     | 2.8247    | 0.6433     | 0.1592  |
| 1.00      | 1.00   | $\infty$ | 1.2997  | $\infty$  | 0.6110    | $\infty$ | 2.9417  | $\infty$  | 2.7305    | 0.6689     | 0.2483  |
| 1.25      | >1     | —        | 1.6324  | —         | 0.8218    | —        | 2.9884  | —         | 3.2049    | 0.7758     | 0.3110  |

Se observa que los valores simulados tienden a ser menores que los teóricos en condiciones de alta congestión (cuando  $\lambda \rightarrow \mu$ ), lo cual es esperable debido a la implementación de una cola de capacidad finita y a la naturaleza finita de la simulación. Además, los tiempos de espera y la longitud de la cola aumentan con el incremento de  $\lambda$ , confirmando el comportamiento esperado del modelo M/M/1.

La probabilidad de denegación de servicio también crece con  $\lambda$ , indicando mayor saturación del sistema. La simulación resulta útil para capturar estas variaciones, especialmente en situaciones en las que el análisis teórico diverge.

### 4.2 Resultados modelo de Inventario

A continuación, se presenta una tabla comparativa con los costos promedio obtenidos de las simulaciones en Python y AnyLogic para las 30 corridas, junto con los valores teóricos esperados calculados para el modelo de inventario ( $s, S$ ).

Table 2: Comparación de Costos Promedio del Modelo de Inventario

| Tipo de Costo                   | Python (Teórico) | Python (Simulación) | AnyLogic (Simulación) |
|---------------------------------|------------------|---------------------|-----------------------|
| Costo de Mantenimiento Promedio | 2007.50          | 1400.28             | 1399.53               |
| Costo de Orden Promedio         | 4461.11          | 2989.00             | 3032.17               |
| Costo de Faltante Promedio      | 811.11           | 1839.67             | 2012.33               |
| <b>Costo Total Promedio</b>     | <b>7279.72</b>   | <b>6228.94</b>      | <b>6444.03</b>        |

### Análisis de los Resultados

Como se puede observar en la Tabla 2, existen diferencias notables entre los costos teóricos y los obtenidos mediante simulación, tanto en Python como en AnyLogic. Estas diferencias son esperadas y reflejan la capacidad de la simulación para capturar la variabilidad y estocasticidad inherente al sistema de inventario, algo que las fórmulas teóricas simplificadas no pueden modelar completamente.

- El **costo de mantenimiento promedio** en ambas simulaciones es significativamente menor que el teórico. Esto sugiere que el inventario promedio real en el sistema simulado es, en efecto, más bajo que la predicción

teórica. Una posible razón es la ocurrencia de faltantes, lo que reduce el inventario disponible y, por ende, el costo de mantenimiento.

- El **costo de orden promedio** también es menor en las simulaciones. Esto podría indicar que, en la práctica, se realizan menos órdenes o la cantidad promedio de cada orden es diferente a la supuesta por el modelo teórico simple.
- La diferencia más marcada se observa en el **costo de faltante promedio**. En ambos casos de simulación (Python y AnyLogic), el costo de faltante es considerablemente **mayor** que el valor teórico. Esto valida la premisa de que la simulación es crucial para sistemas estocásticos: la aleatoriedad en la demanda y los tiempos de entrega provoca más situaciones de escasez de lo que un cálculo teórico lineal podría prever.
- El **costo total promedio** es menor en las simulaciones en comparación con el valor teórico. Esto se debe a la combinación de menores costos de mantenimiento y orden, a pesar del aumento en los costos de faltante. Las pequeñas diferencias entre los resultados de Python y AnyLogic para los costos de simulación son normales y pueden atribuirse a las distintas implementaciones de los generadores de números aleatorios y las lógicas de eventos específicos de cada plataforma.

Estos resultados confirman la utilidad de la simulación como herramienta de análisis, ya que permite obtener una representación más realista del comportamiento de un sistema de inventario bajo condiciones operativas variables.

## 5 Conclusión

En este trabajo se implementaron y analizaron dos modelos fundamentales para la gestión de operaciones en una cervecera: el sistema de colas M/M/1 para el proceso de cocción y el modelo de inventario (s,S) para el control de stock. Mediante simulaciones en Python y AnyLogic, junto con el análisis teórico, se obtuvieron los siguientes hallazgos clave:

- El **modelo M/M/1** demostró que las fórmulas teóricas subestiman los tiempos de espera y rechazos cuando el sistema opera cerca de su capacidad máxima ( $\lambda \rightarrow \mu$ ). Las simulaciones revelaron que con una cola de tamaño 2, la probabilidad de rechazo alcanza hasta 31% cuando  $\lambda = 1.25\mu$ .
- Para el **modelo de inventario**, se comprobó que los costos de faltante simulados (1839.67 en Python, 2012.33 en AnyLogic) fueron significativamente mayores que el valor teórico (811.11), evidenciando que los modelos analíticos simplificados no capturan completamente la variabilidad del sistema real.
- Ambos entornos de simulación (Python y AnyLogic) produjeron resultados consistentes, con diferencias menores al 5% en la mayoría de las métricas, validando así la robustez de las implementaciones.

Estos resultados permiten extraer las siguientes conclusiones prácticas:

- La **simulación por eventos discretos** es esencial para complementar los modelos teóricos, especialmente en sistemas con alta variabilidad como la producción cervecera.
- El **generador de números aleatorios** utilizado (Mersenne Twister en Python) demostró ser adecuado para este tipo de aplicaciones, sin mostrar patrones evidentes que afectaran los resultados.
- La **política (s,S)** implementada requiere ajustes en sus parámetros, particularmente incrementando el punto de reorden (s) para reducir los costosos faltantes detectados.

Este análisis destaca la importancia de validar los modelos teóricos con simulaciones detalladas, especialmente en entornos industriales donde la variabilidad de los procesos puede tener impactos significativos en la operación. Los resultados obtenidos proporcionan una base cuantitativa para optimizar tanto la capacidad de producción como los niveles de inventario en la cervecera estudiada.

## 6 Referencias

- Naylor, T. H. (1982). Técnicas de simulación en computadoras (Cap. 4)
- Law, A. M., & Kelton, W. D. (2000). Simulation Modeling and Analysis (3rd ed.). McGraw-Hill..
- Ross, S. M. (2012). Simulacion. Academic Press.